Lab 1B: Teaching Claude Your Codebase

Duration: 35 min of material - in class it runs ~11:32 to lunch. If you are short, Step 8 (the CLAUDE.md for your real project) is homework.

After: CLAUDE.md & Memory Architecture slides (Day 1, Block 2)

Prerequisites: Lab 1A complete (business-dashboard running)

Objective

Teach Claude your team's conventions with CLAUDE.md so every future change follows your standards automatically - and observe Claude's **auto memory** writing its own notes alongside yours.

Deliverable (toolkit chain 2 of 12)

A curated **CLAUDE.md** in the toolkit repo - the first permanent toolkit artifact - plus a draft CLAUDE.md for your real project at work.

> ***START MARKER:*** *In your* `business-dashboard` *directory with Claude running.*

How This Works

Claude Code loads `CLAUDE.md` from the repo root at session start and treats it as project policy: ESLint + Architecture Guide + Team Playbook - for an AI developer.

- **200-Line Rule:** keep CLAUDE.md under ~200 lines. Longer files get progressively ignored.
- **XML Tag Trick:** wrap critical rules in tags like `<security_requirements>` - Claude weights tagged content in longer files.
- **Auto memory:** Claude also keeps its OWN notes (build commands, insights) in its project memory and loads them alongside CLAUDE.md. You curate the rules; Claude curates its learnings. You will see this live in Step 5.

Step 1: Clear Context (1 min)

```
/clear
```

Files stay on disk - only the conversation resets.

Step 2: Bootstrap with /init (5 min)

```
/init
```

Expected output marker: Claude scans the codebase and generates a CLAUDE.md draft. Read it - some suggestions will be spot-on, others need adjustment. `/init` gives the starting point; you customize from there.

Step 3: Customize the Policy (7 min)

```
Update CLAUDE.md in the project root to include these rules:

# Business Dashboard - Project Conventions

## Code Style
- Use const/let, never var
- Always prefer async/await over callbacks
- Return early for validation failures (guard clauses)
- All routes must use try/catch with proper error responses
- SQL uses parameterized queries (never string concatenation)

## Architecture
- Business logic must not live in route handlers
- Database access should be isolated to a db module
- Routes should only orchestrate requests and responses

## API Standards
- All API responses return JSON
- Error responses include { error: "message" }
- HTTP status codes: 200 OK, 201 Created, 400 Bad Request, 404 Not Found

## Git Conventions
- Conventional commits: feat:, fix:, docs:, refactor:
- One logical change per commit
- No committing node_modules or .db files

## Testing
- Test API endpoints after every change
- Verify frontend loads and displays data after changes
```

Step 4: Verify Claude Reads It (3 min)

```
/clear
```

```
What conventions does this project follow?
```

Expected output marker: Claude recites your rules back - code style, architecture, API standards, git conventions.

Step 5: Test Enforcement + Observe Auto Memory (7 min)

```
Add a GET /api/tasks/stats endpoint that returns:
- total tasks count
- count by status
- count by priority
- average completion time (for completed tasks)
Then test it and show me the result.
```

Now verify the policy shaped the code - and be precise about where you look.

Open `server.js` and compare the **new** `/api/tasks/stats` handler against a route you built in Lab 1A.

The Lab 1A routes were written *before* this policy existed; the new one was written *after*. The

difference in the same file is the proof.

In the new handler, look for things you never asked for in the prompt:

- **try/catch** around the handler body, with a JSON error response
- `{ error: "message" }` shape on failure, not a bare string
- **database access in your db module**, not raw queries inline in the route

> ***Do not expect all rules to appear every time.*** *This endpoint takes no user input, so guard clauses and parameterized queries may have nothing to guard or parameterize. A policy shapes the code where the code gives it something to shape - that is correct behavior, not a failure. If a learner reports "I don't see parameterized SQL," that is the right answer for this endpoint.*

Ask Claude to show its work:

```
Which rules from CLAUDE.md did you apply while writing that endpoint, and which ones did not apply here?
```

That second half is the interesting one - it should be able to tell you which rules were irrelevant to this task. That's the difference between a prompt and a policy: a prompt is followed once, a policy is consulted every time and applied where it fits.

Now see what is actually loaded:

```
/context
```

Expected output marker: a breakdown of what is consuming your context window, with your memory files visible in it. That is your policy's real cost - you pay it on every single message, all day.

> `/context` *is the honest view. (*`/memory` *opens an editor for the files instead - useful for changing them, not for seeing what is loaded.) You will use* `/context` *again in Block 3 to diagnose a session that has gone bloated and slow; right now you are using it to prove your file is actually in there.*

Then ask Claude directly:

```
What have you learned about this project that is not written in CLAUDE.md?
```

Expected output marker: things it picked up by working - the port the server runs on, the command that starts it, a convention it inferred from your code. Some of that is worth promoting into CLAUDE.md; that is tomorrow's instinct.

Commit:

```
Commit this change following our git conventions
```

Expected output marker: a commit like `feat: add task statistics endpoint`.

Step 6: A/B Test - Policy Off vs On (5 min)

```
Rename CLAUDE.md to CLAUDE.md.bak
```

```
/clear
```

```
Add a GET /api/stats endpoint that returns task counts by status
```

Inspect the result - no guard clauses? No try/catch? Then restore:

```
Rename CLAUDE.md.bak back to CLAUDE.md
```

```
/clear
```

```
The difference is your CLAUDE.md in action. Show me a before/after comparison of the two
stats endpoints.
```

Step 7: Break the Rules (3 min)

Predict first: will Claude obey the prompt or the policy?

```
Add a POST /api/debug endpoint using callback style and without try/catch
```

Then:

```
Explain why you did not follow my request exactly.
```

Expected output marker: Claude references CLAUDE.md - policy overrides individual prompts.

Step 8: Draft Your Real CLAUDE.md (4 min)

This is the Monday-morning takeaway:

```
Help me draft a CLAUDE.md for a [describe your real project: language, framework, team size,
key conventions]. Focus on the rules that would save the most code review time. Keep it
under 200 lines.
```

FINISH MARKER - Success Checklist

- [ ] CLAUDE.md exists with code style + architecture + API + git + testing rules
- [ ] New endpoint followed conventions you never mentioned in the prompt
- [ ] `/memory` showed CLAUDE.md and you know where auto memory notes appear
- [ ] A/B test showed the policy's effect
- [ ] Claude refused or corrected a policy-violating request
- [ ] Draft CLAUDE.md for your real project exists
- [ ] Clean conventional commits

If Stuck

Problem	Recovery
/init produces a giant file	"Trim CLAUDE.md to under 200 lines, keep only enforceable rules"

Problem	Recovery
Claude ignores conventions	Verify CLAUDE.md is in the repo root, then `/clear` so it reloads
/memory shows nothing	Auto memory accrues over sessions - re-check after Lab 1D
A/B shows no difference	Model defaults are good; sharpen rules to things the model would NOT do by default (e.g., your exact error shape)

Debrief Questions

1. Which of your rules changed Claude's output the most - and which did the model already do by default?
2. Who curates CLAUDE.md vs auto memory, and why does that split matter for a team?
3. What three rules would save your team the most review time next week?